

CHAPITRE III

les systèmes d'exploitation embarqués

I. Définitions

I. Définitions

Le système d'exploitation (SE) ou Operating System (OS), est l'ensemble de programmes central d'un système qui sert d'interface entre le matériel et les logiciels applicatifs.

Il offre une large gamme de services utilisés par les applications :

- Il facilite l'exploitation des périphériques matériels dont il coordonne et optimise l'utilisation
- Il met à disposition des logiciels et une interface de programmation standardisée d'utilisation des matériels
- Il réalise différentes fonctions visant à assurer la fiabilité (tolérance aux pannes, isolation des fautes) et la sécurité informatique (traçabilité, confidentialité, intégrité et disponibilité).

I. Définitions

Un système d'exploitation peut servir à :

- Coordonner l'utilisation du ou des processeur(s), et accorder un certain temps pour l'exécution de chaque processus
- Réserver de l'espace dans les mémoires pour les besoins des programmes et à organiser le contenu des disques durs ou d'autres mémoires de masse en fichiers et répertoires.
- Créer l'image numérique qui contiendra les interfaces homme-machine des différents programmes et à l'IHM ainsi qu'à recevoir les manipulations effectuées par l'utilisateur via celle-ci.

I. Définitions

En résumé, Un système d'exploitation :

- Fait le lien entre le logiciel (application) et le matériel
- Abstrait certaines caractéristiques du matériel
- Fournit des services communs (accès aux ressources, synchronisation, gestion de fichier)
- Offre des possibilités de tests et de traces

I. Définitions

Généralement, un système embarqué :

- Dispose de ressources limitées
- Ne possède pas toujours de système de fichiers
- Doit être le moins cher possible
- Ne doit pas consommer d'énergie inutilement

Dans une approche minimaliste, le système d'exploitation :

- Initialise le matériel
- Donne la main à l'application

I. Définitions

Dans une approche maximaliste, le système d'exploitation :

- Fournit tous les services d'un ordinateur de bureau
- Fournit des machines virtuelles
- Fournit des bibliothèques graphiques

En général, le système d'exploitation fournit les services :

- De gestion de temps
- D'accès au réseau
- De gestion de la mémoire

I. Définitions

Avant de choisir son système, il faut définir :

- Le prix de revient maximum de chaque unité
- Le temps de réponse aux actions de l'utilisateur
- La capacité totale de traitement
- La disponibilité et la possibilité de le modifier
- Le support disponible et la fiabilité du fournisseur

I. Définitions

Contrairement aux logiciels classiques, les logiciels embarqués ont une durée de vie particulièrement longue ce qui implique une importance de faire évoluer le logiciel indépendamment des aléas économiques.

On trouve des systèmes d'exploitation dans des ordinateurs, mais aussi dans des voitures (environ 60 CPU / voiture), appareil photo, routeur ADSL, chaîne HIFI...

II. Avantage d'un système d'exploitation embarqué

II. Avantage d'un système d'exploitation embarqué

Un système d'exploitation embarqué permet :

- D'avoir un gain de temps de développement : utilisation des APIs de l'OS, donc code à ne pas développer, facilité à utiliser les ressources du système.
- Donne la possibilité de bénéficier des mêmes avancées technologiques que les applications classiques (TCP/IP, HTTP, etc.)
- D'utiliser un Environnement de développement plus performant

III. Inconvénient d'un système d'exploitation embarqué

III. Inconvénient d'un système d'exploitation embarqué

Par contre un système d'exploitation embarqué :

- Utilise trop de mémoire
- Utilise trop de puissance du processeur
- Peut être cher
- Est trop compliqué à utiliser

IV. Critères de choix pour un système d'exploitation embarqué

IV. Critères de choix pour un système d'exploitation embarqué

Lors du choix d'un système d'exploitation embarqué, les critères suivant interviennent :

- Le besoin de performance
- Les outils de développement disponible
- Royalties
- Le prix d'achat
- L'espace mémoire nécessaire
- Les processeurs supportés
- La simplicité d'utilisation
- La compatibilité avec les applications
- Les services offerts
- La compatibilité avec d'autres systèmes d'exploitation
- Open-Source

V. Typologie

V. Typologie

Chaque système d'exploitation est conçu pour fonctionner avec une gamme particulière de machines (**type de processeur, constructeur, architecture**). Si un système d'exploitation est disponible pour plusieurs gammes machines différentes, alors le même code source est compilé [] et adapté à chaque gamme de machine. La liste exacte des machines sur lesquelles il peut être utilisé dépend aussi de la palette de pilotes inclus dans le système d'exploitation.

Selon sa construction, un système d'exploitation peut être multitâches ou alors mono-tâche, multi-utilisateurs ou alors mono-utilisateur, il peut aussi être multiprocesseurs et/ou en temps réel.

V. Typologie

- Multitâches : permet l'exécution simultanée de plusieurs programmes. Les premiers systèmes d'exploitation multitâches sont apparus dans les années 1960. Tous les systèmes d'exploitation contemporains sont multitâches.
- Multi-utilisateurs : conçu pour être utilisé **simultanément par plusieurs usagers**, souvent à travers un réseau informatique. De tels systèmes d'exploitation sont typiquement utilisés pour des serveurs ainsi que des **mainframes** (Ordinateur central de grande puissance de traitement.) ou des **superordinateurs** (HPC : High Performance Computing). Ils sont multitâches et en général sécurisés, c'est-à-dire qu'ils vont refuser d'exécuter toute opération pour laquelle l'utilisateur n'a pas préalablement reçu la permission.

V. Typologie

- **Multiprocesseurs** : conçu pour exploiter un ordinateur équipé de plusieurs processeurs. Dans de tels systèmes d'exploitation, plusieurs programmes sont exécutés simultanément par les différents processeurs.
- **Temps réel** : garantit que les opérations seront effectuées en respectant des délais très stricts, et ce quelles que soient les conditions d'utilisation. De tels systèmes d'exploitation sont utilisés dans l'industrie, l'aéronautique ou l'électronique grand public pour créer des systèmes temps réel.

VI. Historique

VI. Historique

Date	Descriptif
1940	Programmation des ordinateurs par manipulation d'interrupteurs à bascule, puis plus tard par cartes perforées
1940-1960	Les ordinateurs sont très coûteux et appartiennent à des entreprises et des institutions. Chaque utilisateur avait le droit d'utiliser l'ordinateur pendant un temps limité. Les ordinateurs sont Mono-tâches et mono-utilisateurs. Naissance de l'idée d'isoler dans des applications séparées, les instructions routinières et stocké dans la mémoire. Apparition du 1er système d'exploitation (sous la forme la plus simple).
1960	Apparition de la multiprogrammation Périphérique des ordinateurs des années 60 : un lecteur de carte perforée, une imprimante et un disque dur. Constatation du temps perdu à attendre la fin d'exécution des périphériques. Ce qui implique l'élaboration des premiers systèmes d'exploitation multiprogrammation qui permettent l'exécution de plusieurs programmes à la fois. Multiprogrammation : chargement en mémoire simultanément des programmes. Le premier programme est exécuté jusqu'à être en attente de libération d'un périphérique, le système d'exploitation enchaîne sur l'exécution du deuxième programme, puis le troisième et ainsi de suite. Technique précurseur du multitâche.

VI. Historique

Date	Descriptif
1965	Création du premier système d'exploitation multitâches et multi-utilisateurs : Multics (MULTiplexed Information and Computing Service ou service multiplexé d'information et de calcul) crée par le MIT (Massachusetts Institute of Technology). Sur le principe de la multiprogrammation, le système d'exploitation autorise le chargement de plusieurs programmes en mémoire et gère le passage de l'un à l'autre, mais cette fois-ci sans attendre le blocage d'un programme. Chaque programme est exécuté pendant une durée de quelques millisecondes ce qui donne une illusion que les programmes s'exécute simultanément. Ces programmes peuvent appartenir à plusieurs utilisateurs distincts. Multics fut un échec commercial dut à sa complexité, alors qu'il permettait un très bon retour sur investissement (1 ordinateur pour plusieurs utilisateurs)
1969	Ken Thompson créât Unics (UNiplexed Information and Computing Service), puis finalement baptisé UNIX, une version allégé de Multics afin de tourner sur des machines moins couteuses. Rapidement reprogrammé dans un langage de programmation plus approprié (le C, développé par Ritchie pour l'occasion), UNIX se révèle particulièrement simple à porter sur de nouvelles plateformes, ce qui assure son succès.
1972	Sortie du premier micro-ordinateur, le R2E, équipé d'un microprocesseur Intel 8008 et du système d'exploitation SYSMIC - appelé alors moniteur d'exploitation. le système d'exploitation SYSMIC sera plus tard renommé PROLOGUE lors du rachat de la société par Bull en 1978.

VI. Historique

Date	Descriptif
1975	Sorti du premier micro-ordinateur accessible aux particuliers, l'Altair 8800 (prix : 400 \$), mais avec une utilisation compliquée. Gary Kildall, ingénieur pour Intel crée CP/M, le premier système d'exploitation pour micro-ordinateur. Création de la société Microsoft par Bill Gates et Paul Allen.
1980	Bill Gates rachète QDOS – un système d'exploitation quick-and-dirty pour les processeurs Intel 8086 – pour proposer à IBM le package DOS/BASIC. Après quelques modifications effectuées à la demande d'IBM, le système est baptisé MS-DOS
1983	Richard Stallman du Massachusetts Institute of Technology lance l'idée d'un système d'exploitation sous licence libre : GNU (GNU's Not UNIX). Il développe des outils de programmation, des logiciels utilitaires, et crée la GNU General Public License - un contrat de licence qui autorise la publication du code source.
1991	Sortie de la version 1ère version de Linux développé de Linux par Linus Torvalds, étudiant de l'université d'Helsinki.
1992	Linux passe sous licence GNU
2000	1ers systèmes d'exploitation embarqués pour PDA, portable...

VII. Composition

VII. Composition

Un système d'exploitation est composé d'une large palette de programmes. La composition exacte dépend de l'usage cible et du type d'appareil informatique auquel le système est destiné (ordinateur personnel, serveur, superordinateur ou encore système embarqué).

Exemple :

- Si le système d'exploitation est destiné à être utilisé sur un ordinateur personnel ou une console de jeu vidéo, l'interface graphique sera raffinée et ergonomique.
- S'il est destiné à être utilisé sur un serveur, il comprendra une large palette de protocoles et de pilotes pour du matériel réseau, sera multitâches et muni de contrôles d'accès.
- Au contraire, le nombre de pilotes sera restreint au minimum et le système d'exploitation sera prévu pour être enregistré sur une mémoire morte s'il est destiné à un assistant personnel ou un téléphone portable.
- Un système d'exploitation destiné à des superordinateurs sera massivement multiprocesseur, c'est-à-dire qu'il pourra être utilisé sur un ordinateur équipé de centaines voire de milliers de processeurs.

VII. Composition

1) Interface de programmation

Le système d'exploitation effectue de nombreuses actions sur demande des logiciels applicatifs. Le point de contact entre un logiciel applicatif et le système d'exploitation est appelé **interface de programmation** (API : Application Programming Interface). Ce peut être des fonctions mises à disposition dans des **bibliothèques logicielles**, ou des **logiciels serveur** qui répondent aux **requêtes** envoyées par les logiciels applicatifs.

L'utilisation de la même interface de programmation quel que soit le matériel, le protocole ou le système de fichier concerné assure la **portabilité** des logiciels applicatifs : un logiciel applicatif donné pourra fonctionner sur différents systèmes, quelle que soit leur configuration, en particulier quel que soit le matériel, le système de fichier ou le protocole utilisé.

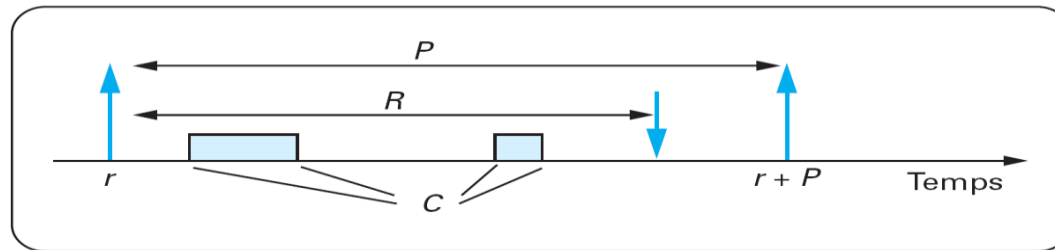
POSIX (**Portable Operating System Interface**) est une norme relative à l'interface de programmation du système d'exploitation. De nombreux systèmes d'exploitation sont conformes à cette norme, notamment les membres de la famille Unix.

VII. Composition

Caractérisation des tâches

Les tâches peuvent être **périodiques** ou **apériodiques**, à contraintes temporelles **strictes** ou **relatives**. On définit un modèle canonique regroupant les principaux paramètres temporels qui caractérisent chaque tâche temps réel afin d'aider au paramétrage de l'ordonnanceur temps réel :

VII. Composition



Une tâche est définie par :

- r Date de déclenchement de la requête
- C Durée d'exécution
- R Délai critique
- P Période de la tâche

↑ Déclenchement
↓ Échéance $r + R$

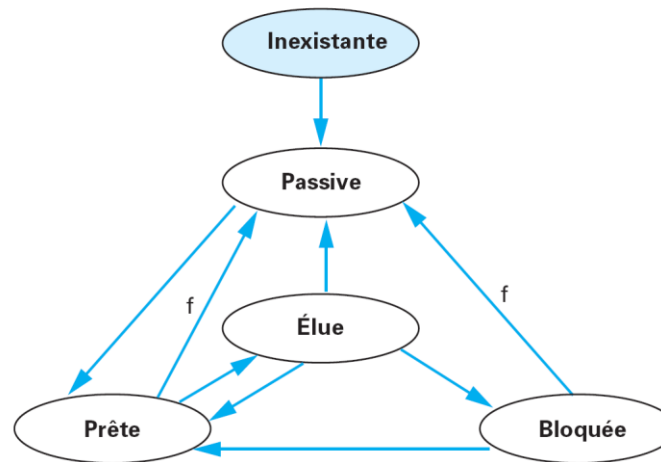
$$0 \leq C \leq R \leq P$$

Modèle canonique d'une tâche

Dans le cas de tâches à contraintes temporelles relatives, le paramètre R est parfois omis. Lorsque la tâche est apériodique, le paramètre P n'est pas donné. La détermination de ces paramètres avec le plus d'exactitude possible permet d'obtenir un bon ordonnancement (à faible risque d'erreur). Pour obtenir un ordonnancement fiable, il est nécessaire d'inclure dans l'évaluation de ces paramètres les temps de commutation des tâches, les durées d'utilisation des primitives du système d'exploitation, les durées de prise en compte des interruptions et les durées d'exécution de l'ordonnanceur.

VII. Composition

Le partage du processeur et des ressources entre les différentes tâches nécessite de définir les états possibles suivants pour chaque tâche :



f : cas où la requête est interrompue à son échéance

Les tâches périodiques sont déclenchées à la date de réveil des requêtes successives et redeviennent **passives** à chaque fin d'exécution.

Lorsqu'une tâche est **élue**, le processeur lui est alloué et l'exécute. Lorsqu'une tâche est **bloquée**, elle est en attente d'une ressource, d'un message ou d'un signal de synchronisation. Lorsqu'une tâche est **prête**, elle attend d'être élue.

VII. Composition

Remarques :

- On dit qu'une tâche est **préemptible** lorsqu'elle peut être arrêtée et remise dans l'état prête afin de réquisitionner le processeur au profit d'une autre tâche. Au contraire, une tâche est non préemptible lorsqu'elle ne doit plus être interrompue après avoir été élues. C'est le cas des tâches qui s'exécutent sous le contrôle du mécanisme d'interruption ou des tâches gérant des transferts directs sur un bus sans passer par un contrôleur DMA.
- Certaines tâches peuvent être **dépendantes** entre-elles. Les tâches peuvent interagir selon un ordre partiel prédéterminé ou induit par la communication de messages ou par une relation explicite de synchronisation. Il peut alors en résulter une relation de précédence entre tâches. Cette relation est dite statique, car elle est connue a priori et n'évolue pas. Elle est représentée par un graphe de dépendance.
- Priorité externe : on introduit parfois à la conception une priorité fixe dite priorité externe. C'est une forme primitive d'ordonnancement temps réel où tout est déterminé dès la conception par un ordonnancement hors ligne ou par des règles qui imposent un ordre a priori. Par exemple, la tâche de gestion d'horloge ou la tâche de sauvegarde en cas de chute de courant sont à exécuter toutes affaires cessantes.

VII. Composition

- Gigue maximale : l'exécution de tâches périodique est parfois soumise à des contraintes de régularité pour le début de leur exécution. C'est le cas dans l'acquisition de données par échantillonnage périodique, ou dans une régulation par PID ou dans l'envoi continu de son et d'images pour le multimédia. La variation entre le début d'exécution de deux requêtes successives est appelée gigue et on définit pour ces applications une gigue maximale à ne pas dépasser : $-\max \leq (s_{i+1} - (s_i + P)) \leq \max$

VII. Composition

Notion de processus

Un processus est défini par les caractéristiques suivantes :

- c'est une entité dynamique correspondant à l'exécution séquentielle d'un ou plusieurs programmes constituant des tâches
- il peut être créé, exécuté ou détruit
- il est caractérisé par son contexte :
 - l'état des pointeurs de zones mémoire :
 - la zone de programme (le compteur programme PC)
 - La zone de données (le TAS ou HEAP)
 - la pile
 - les caractéristiques du processus
 - le nom d'identification du processus par le système d'exploitation
 - le niveau de priorité du processus
 - les droits d'accès aux ressources matérielles et logicielles par le processus
 - un identificateur d'état du processus : suspendu, prêt, en attente, actif

L'ensemble du contexte d'un processus constitue le descripteur de processus. Le système d'exploitation tient à jour une table des descripteurs.

VII. Composition

L'ordonnanceur

L'ordonnanceur (**scheduler**) est le programme du système d'exploitation qui contrôle **le déroulement des autres programmes**. Dans un système d'exploitation multitâche l'ordonnanceur assure l'exécution simultanée de plusieurs programmes (**concurrency**).

L'exécution simultanée de plusieurs programmes est réalisée de la manière suivante: l'ordonnanceur commande l'exécution à tour de rôle de chaque programme par le ou les processeurs du système, chaque processeur exécute les instructions d'un programme pendant une fraction de seconde, puis l'ordonnanceur effectue une commutation de contexte (context switch), suspend l'exécution du programme en cours et passe au programme suivant.

VII. Composition

Quel processus doit choisir l'ordonnanceur dans la file d'attente des processus prêts ?

Un bon algorithme d'ordonnancement doit être capable de :

- S'assurer que chaque processus reçoit sa part du temps CPU : équité
- Gérer correctement les priorités s'il y en a.
- Utiliser le temps processeur à 100% : efficacité
- Minimiser le temps de réponse en mode interactif : temps de réponse
- Minimiser le temps d'attente en traitement par lots : temps d'exécution
- Maximiser le nombre de travaux effectués en une heure : rendement

VII. Composition

Il existe 3 types d'ordonnancement :

- Ordonnancement sans réquisition, type 1
 - ✓ Exécution jusqu'à terminaison d'un processus
 - ✓ Adapté aux systèmes à traitement par lot
 - ✓ Non adapté aux systèmes interactifs et multi-utilisateurs
- Ordonnancement sans réquisition, type 2 (Collaborative Scheduling)
 - ✓ Exécution jusqu'à ce qu'un processus "passe la main"
 - ✓ Oblige le développeur à insérer des `yield()` (windows 3.11, le code MSDOS dans win9x)
- Ordonnancement avec réquisition (Preemptive Scheduling) : OS modernes Suspension du processus en exécution par :
 - ✓ Appels systèmes (points de préemption dans le code de la fonction)
 - ✓ Interruptions matérielles (prioritaires)
 - ✓ Expiration du quantum de temps (IT d'un timer armé par l'OS)

VII. Composition

2 algorithmes utilisés pour l'ordonnancement :

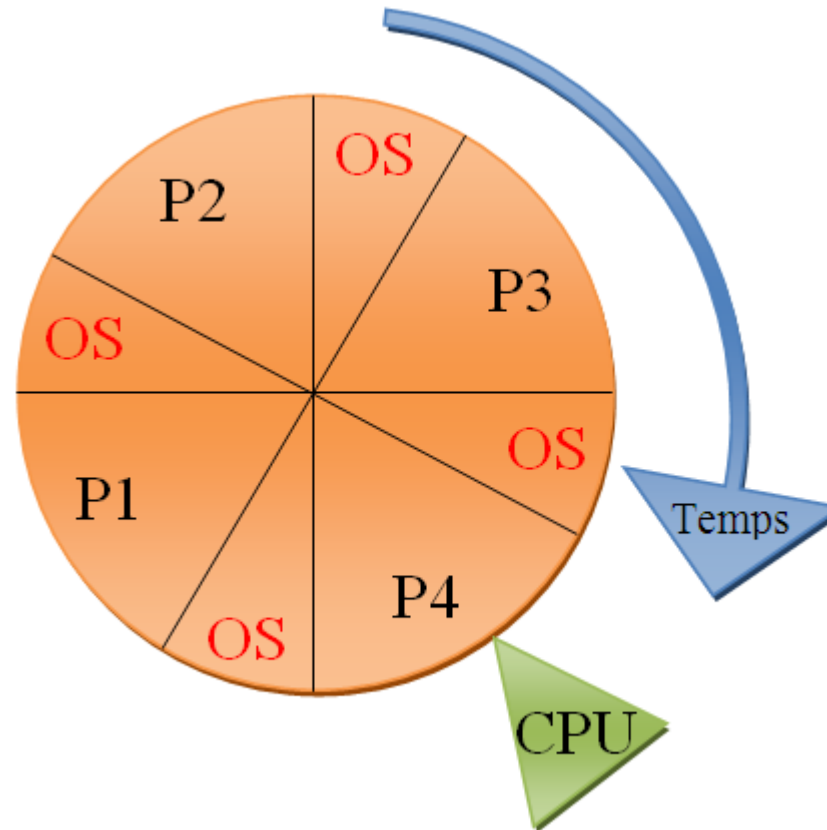
- Méthode du tourniquet.

Les processus prêts se succèdent et sont exécutés sur le CPU pendant un Quantum de temps calculé en fonction de la charge du système. L'OS assure le changement de contexte entre 2 quotas de temps.

Durée du quantum :

- ✓ Un quantum trop petit provoque trop de commutations de processus. Le changement de contexte devient coûteux.
- ✓ Un quantum trop élevé augmente le temps de réponse

VII. Composition



VII. Composition

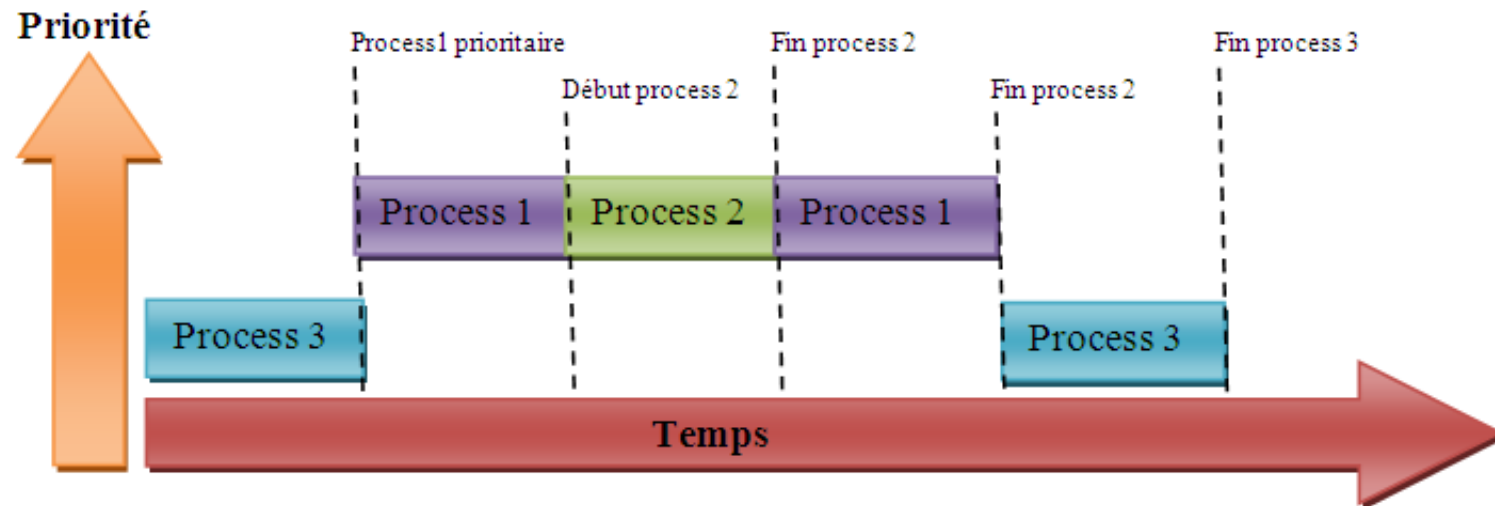
- Par file de priorité

Les processus prêts de forte priorité s'exécutent d'abord

Lorsque la file d'une priorité est vide on passe à la file de priorité inférieure

A l'intérieur d'une file on peut utiliser l'algorithme du tourniquet.

Si un process de basse priorité utilise une ressource utile à un process de haute priorité, l'ordonnanceur peut « inverser » les priorités pour libérer la ressource.



VII. Composition

3) La gestion des interruptions

La gestion des interruptions permet la gestion de plusieurs périphériques: minuterie (timer), moteurs, capteurs, disques, etc, via des requêtes asynchrones signalés par des interruptions

Il existe 2 types d'interruptions :

- Interruptions matérielles
- Interruptions logicielles

Le code exécuté lors d'une interruption est dicté par le CPU à l'aide du vecteur d'interruption. Mais l'OS intervient pour:

- Connecter une adresse mémoire à chaque ligne d'interruption
- Que faut-il faire après avoir servi une interruption

Les interruptions permettent bien évidemment la gestion de l'aspect temps réel.

VII. Composition

4) La communication interprocessus

Les systèmes d'exploitation sont typiquement composés de nombreux programmes qui coopèrent en s'échangeant des informations. Les échanges d'informations entre les processus (programmes en cours d'exécution) peuvent se faire à l'aide **de messages stockés en mémoire**, le système d'exploitation s'occupant de collecter les messages et les mettre à disposition de chaque destinataire. Tout processus qui attend un message sera interrompu jusqu'à l'arrivée d'un message et l'ordonnanceur évitera toute commutation de contexte vers un processus en attente.

Le système d'exploitation s'assure également que les processus qui partagent un espace de stockage commun (en lecture ou écriture) ne se trouvent pas en situation de **compétition**, c'est-à-dire dans une situation où le résultat d'une opération dépend de l'ordre dans lequel les processus accèdent à l'espace partagé.

VII. Composition

Il existe plusieurs concepts :

- La mémoire partagée : Une zone de mémoire est partagée par plusieurs processus. Elle est vue comme un tableau. Il n'y a pas de mécanisme de synchronisation.
- Les fichiers mappés en mémoire : C'est une zone de la mémoire partagée par plusieurs processus. Elle est vue comme un fichier. Il n'y a pas de mécanisme de synchronisation.
- Les sockets : Permet d'envoyer des informations à travers un réseau.
- Les RPC (Remote Procedure Call) : Permet à un processus d'appeler une fonction sur une autre machine. L'interface permet de masquer les différences entre machines telles que la taille des entiers...
- DCOM (Distributed Component Object Model) : Permet de manipuler les données des objets à travers un réseau en faisant des appels aux méthodes des objets à distance.

VII. Composition

Les systèmes d'exploitation proposent plusieurs moyens de protection pour la synchronisation des processus et des threads :

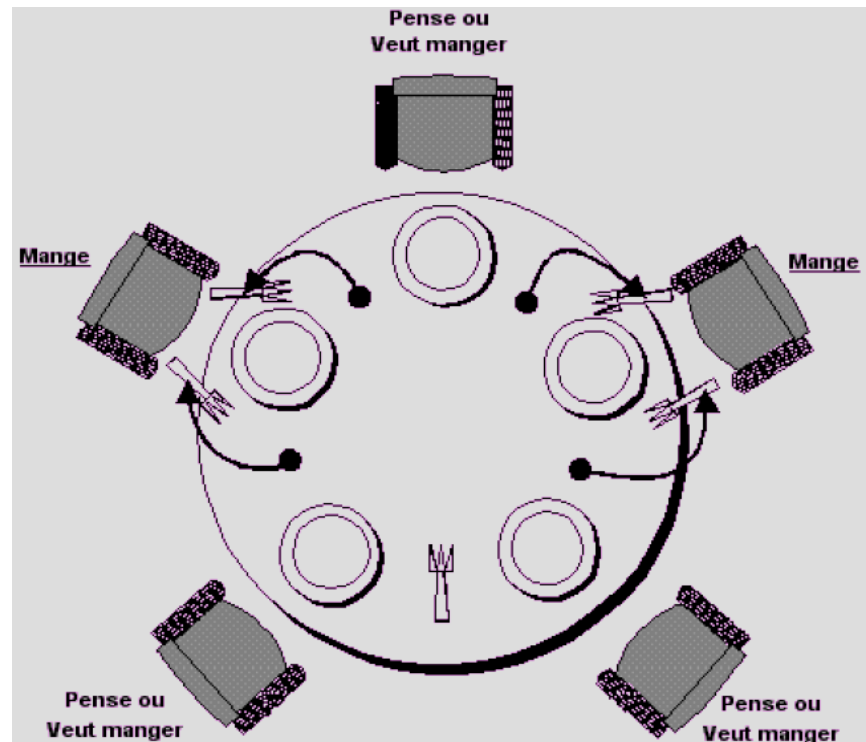
- Critical Section : **uniquement entre thread (léger en mémoire)**
- Mutex : **entre processus ou entre thread**
- Sémaphore : **entre processus ou entre thread (permet le comptage automatique du nombre d'accès)**
- **pthread_mutex pour les threads POSIX**

VII. Composition

Exemple: Sémaphore (Problème des philosophes)

Un philosophe peut avoir 3 états

- Pense (pendant un temps aléatoire)
- A faim (essaye de prendre 2 fourchettes)
- Mange (pendant un tps aléatoire)



VII. Composition

On veut éviter 2 problèmes qui illustrent des situations de concurrence sur le partage de ressources entre processus.

- L'interblocage : chaque philosophe a pris une fourchette et attend que la 2ième soit libérée.
- La famine : un philosophe ne pouvant pas manger repose sa fourchette et attend. Un cas critique ayant une faible chance de se produire est celui où les philosophes attendent le même temps à chaque fois...

Résolutions :

- Résolution (1) par un sémaphore : avant de prendre les 2 fourchettes, manger puis rendre les fourchettes, un philosophe prend un sémaphore et le rend à la fin des 5 opérations. Pb : un seul philosophe mange à la fois.
- Résolution (2) par 6 sémaphores (un de test et un par philosophe) : chaque philosophe qui a faim prend le sémaphore de test et vérifie qu'il peut manger. S'il ne peut pas, il libère le sémaphore de test et bloque sur son sémaphore. Quand un philosophe a fini de manger, il teste si ses voisins peuvent manger et libère leur sémaphore dans ce cas.

VII. Composition

Exemple concret : Afficheur LCD

Un microcontrôleur pilote un afficheur LCD en lui envoyant des octets correspondant :

- Un caractère à afficher à la position courante du curseur
- Un ordre de déplacement spécifiant la ligne et la colonne

On souhaite que plusieurs tâches puissent afficher à des endroits différents de l'écran.

On utilise les entités suivantes :

- Une file d'attente contenant les données à envoyer à l'afficheur LCD
- Une tâche recevant successivement les octets de la file d'attente et les envoyant à l'afficheur
- Un sémaphore permettant un accès exclusif à la file d'attente, pour que le remplissage se fasse de manière cohérente
- D'autres tâches prenant le sémaphore, mettant les données dans la file d'attente et relâchant ensuite le sémaphore

VII. Composition

Ainsi, les différentes chaînes de caractère à afficher ne peuvent pas se mélanger.

Si la tâche qui envoie les données à l'afficheur est plus prioritaire que la tâche qui souhaite afficher quelque chose, l'affichage peut commencer dès l'entrée du premier octet dans la file d'attente et n'être limité que par les limitations de l'afficheur lui-même.

Si les priorités sont égales, il est conseillé de donner la main au consommateur, afin qu'il vide la file d'attente et limite les inversions de priorité par la suite.

5) Gestion de la mémoire

3 termes à connaître pour la gestion de la mémoire :

- Allocation: Allouer à chaque tâche la mémoire dont elle a besoin
- Mapping: Faire la correspondance entre la mémoire physique et l'adressage utilisé par les tâches.
- Protection: Etablir un ensemble de comportements à adopter lorsqu'une tâche utilise de la mémoire non allouée.

VII. Composition

6) La mémoire virtuelle

Le mécanisme dit de mémoire virtuelle est destiné à simuler la présence ou l'absence de mémoire centrale par manipulation de l'unité de gestion mémoire (MMU : **Memory Management Unit**).

La mémoire virtuelle permet d'exécuter simultanément plus de programmes que ce que la mémoire centrale peut contenir. Chaque programme n'ayant pas besoin que la totalité des informations qu'il manipule soit présent dans la mémoire centrale (ni le code, ni les données, ni la pile n'ont besoin d'être totalement en mémoire centrale), une partie des informations sont stockées dans **la mémoire de masse** (en général dans un fichier ou une partition de disque dur) habituellement plus importante mais plus lente et sont transférées en mémoire centrale lorsque le programme en a besoin.

VII. Composition

Les programmes disposent d'un (ou plusieurs) **espaces virtuels de mémoire continus** pour travailler. Les adresses des données sont dites virtuelles dans la mesure où l'information adressée ne se trouve pas forcément en mémoire centrale, et pas forcément à l'adresse indiquée. Lorsque le programme essaie de lire ou écrire une donnée dans sa mémoire virtuelle, l'unité de gestion de mémoire cherche l'adresse physique correspondant à l'adresse virtuelle sollicitée grâce à une table de correspondance. Si l'emplacement n'est pas présent en mémoire centrale, il n'y aura évidemment aucune adresse physique correspondante. Le système d'exploitation devra alors chercher à libérer un espace en mémoire centrale en échangeant (swap) le contenu d'un emplacement donné de mémoire centrale avec le contenu sollicité, qui se trouve en mémoire de masse. Cette opération s'effectue automatiquement, à l'insu des programmes.

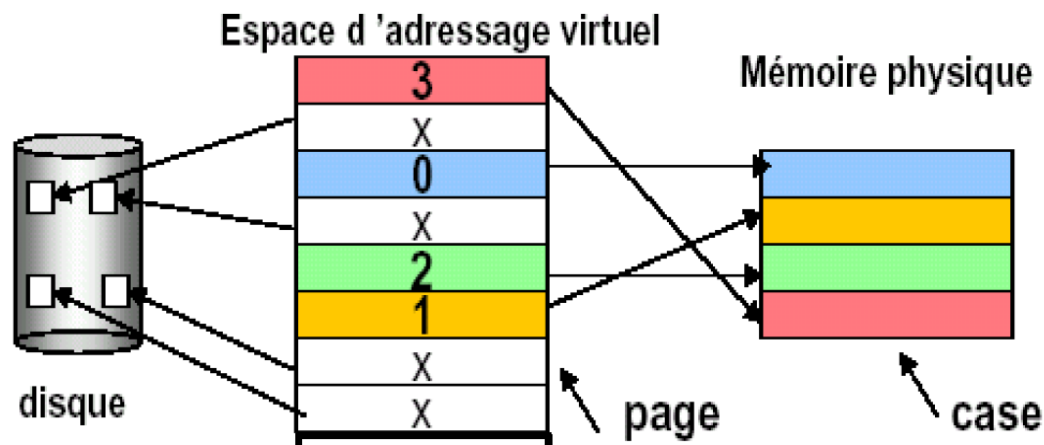
VII. Composition

Le code, les données et la pile d'un processus sont séparés dans des segments différents (**protection des segments entre eux**).

Les segments sont découpés en tranches. La mémoire physique est elle aussi découpée en tranches de même taille.

Le programme "voit" un adressage plan virtuel, toujours le même. Les adresses virtuelles sont traduites en adresses physiques par le Memory Management Unit (**unité spécialisée du processeur**).

La CPU connaît par la table des segments l'adresse de début et de fin de chacun des segments pour chaque processus ainsi que le privilège associé (**mode noyau ou utilisateur**).



VII. Composition

Certaines pages ont été déchargées sur le disque (parce que la mémoire physique est trop petite), d'autres sont en mémoire physique, et certains espaces sont laissés vides (réservé à l'allocation dynamique...)

Des mémoires associatives, incorporées dans l'unité de gestion de mémoire, accélèrent le calcul des adresses. Les systèmes d'exploitation utilisent généralement deux mémoires associatives : une pour le mode kernel et une pour le mode utilisateur. La mémoire du mode kernel est arrangée de manière à permettre au processeur d'utiliser la totalité de la mémoire centrale disponible - lors de l'exécution des programmes du noyau du système d'exploitation. Tandis que celle du mode utilisateur est arrangée de manière à protéger le noyau (qui est ainsi invisible pour le programme en question) - lors de l'exécution des programmes hors du noyau.

VII. Composition

Chaque programme dispose de sa propre table de correspondance, ce qui permet de les isoler les uns des autres. Lors d'une commutation de contexte, le système d'exploitation placera la table du programme courant dans la mémoire associative. Le système d'exploitation crée également de nouvelles tables pour les programmes qui démarrent et décide quels emplacements de mémoire virtuelle seront ou ne seront pas présents en mémoire centrale.

VII. Composition

7) Pilotes

Un pilote (**driver**) est un programme qui contient les instructions à exécuter pour utiliser un certain périphérique. Les pilotes sont fournis par l'auteur du système d'exploitation ou le fabricant du périphérique. Un système d'exploitation contient typiquement de nombreux pilotes correspondant à de nombreux composants matériels susceptibles d'être utilisés.

L'interface de programmation est similaire sur tous les pilotes, tandis que les instructions du programme diffèrent en fonction des caractéristiques de chaque composant matériel, ce qui assure l'utilisation banalisée du périphérique.

Le système d'exploitation dirige **l'utilisation des pilotes par les logiciels applicatifs**. Il intervient lorsque plusieurs logiciels applicatifs tentent d'utiliser le même matériel en même temps, et **active** ou **désactive** les pilotes en fonction du matériel présent.

VII. Composition

Lors de chaque événement survenu sur un périphérique (réception d'informations, fin d'un envoi, manipulation effectuée par l'utilisateur...), ce dernier provoque une **interruption matérielle**, un mécanisme électronique qui provoque une **commutation de contexte** ; le système d'exploitation déclenche alors l'exécution des instructions du pilote concerné.

Les logiciels applicatifs ainsi que le système d'exploitation peuvent également, au besoin, **déclencher l'exécution des instructions d'un pilote**.

VII. Composition

8) Système de fichiers

Un système de fichier (FS : **File System**) est une structure en arbre dans laquelle sont stockés des fichiers ainsi que d'autres informations. Divers programmes du système d'exploitation s'occupent de répartir les fichiers sur les mémoires de masse et de les relier pour former la structure du système de fichiers.

Il existe diverses manières de disposer les fichiers dans une mémoire de masse. Une mémoire de masse qui a été remplie avec un système d'exploitation selon une disposition donnée sera reconnue par un autre système d'exploitation uniquement si **celui-ci reconnaît la disposition en question**.

VII. Composition

Un système d'exploitation contient typiquement différents programmes qui servent à manipuler un système de fichier. Chaque programme manipule les fichiers conformément à **une disposition donnée**. Ces programmes effectuent des opérations telles que créer des fichiers, les copier, les supprimer, créer et supprimer des répertoires ainsi que formater la mémoire de masse c'est-à-dire créer une structure vide.

Dans un système d'exploitation multi-utilisateurs, les programmes qui manipulent le système de fichier effectuent des contrôles pour vérifier qu'aucun fichier n'est manipulé par une personne non autorisée. Le système d'exploitation refusera toute manipulation non autorisée.

VII. Composition

Un système de fichiers embarqué est utile pour :

- Stocker des données de tailles connues ou inconnues de manière persistante ou temporaire
- Stocker des informations de configuration
- Stocker des programmes
- Echanger des informations avec l'extérieur, sur un support interne ou un support amovible

On peut souhaiter ne pas utiliser de système de fichiers quand:

- Les données sont peu nombreuses
- Elles sont plus efficacement stockées dans des blocs de taille et d'adresse fixes
- La complexité d'un système de fichiers ne se justifie pas

VII. Composition

a) Le format FAT

La famille de formats FAT (file allocation table) est la plus couramment utilisée pour les fichiers échangés avec le monde extérieur :

- Simple à mettre en œuvre
- Extrêmement répandue
- Protégée par des brevets logiciels de Microsoft qui les exerce (à l'encontre de TomTom) depuis février 2009

Plusieurs bibliothèques libres (par exemple FatFs) implémentent ces formats et peuvent être intégrées dans les applications si la présence de ces formats est souhaitée.

VII. Composition

Les formats FAT sont peu adaptés à un stockage immédiat sur mémoire flash :

- Absence de système de vérification d'intégrité
- Absence de système de répartition d'écriture (wear levelling, temps d'effacement des blocs réécrits surtout sur flash NOR)
- Favorise un système de stockage linéaire des blocs, où aller d'un bout à l'autre du disque prend du temps, ce qui n'est pas le cas avec la mémoire flash

On peut soit utiliser un format de stockage adapté, soit insérer un contrôleur (logiciel ou matériel) intermédiaire qui :

- Efface les blocs inutilisés lorsqu'il n'a rien à faire
- Stocke des informations de contrôle permettant de détecter des erreurs de lecture ou d'écriture (vérification a posteriori)
- Répartit les écritures pour équilibrer le nombre d'écritures sur chaque bloc.
- De tels contrôleurs sont inclus dans la plupart des clés USB et des disques flash

VII. Composition

b) Le format NTFS

Le système de fichiers NTFS (**New Technology File System**) utilise un système basé sur une structure appelée « **table de fichiers maître** », ou MFT (**Master File Table**), permettant de contenir des informations détaillées sur les fichiers. Ce système permet ainsi l'utilisation de noms longs, mais, contrairement au système FAT32, il est sensible à la casse, c'est-à-dire qu'il est capable de différencier des noms en majuscules de noms en minuscules.

Pour ce qui est des performances, l'accès aux fichiers sur une partition NTFS est **plus rapide** que sur une partition de type FAT car il utilise un arbre binaire performant pour localiser les fichiers. La limite théorique de la taille d'une partition est de 16 exaoctets (17 milliards de To), mais la limite physique d'un disque est de 2To (voir plus actuellement).

VII. Composition

C'est au niveau de la sécurité que NTFS prend **toute son importance**, car il permet de définir des attributs pour chaque fichier. La version 5 de ce système de fichiers (en standard sous Windows 2000 alias NT 5) amène encore de nouvelles fonctionnalités parmi lesquelles des performances accrues, des quotas de disque par volume définis pour chaque utilisateur. NTFS v.5 devrait aussi apporter la possibilité d'administration à distance...

VII. Composition

c) Le format YAFFS2

Des systèmes de fichiers adaptés aux mémoires flash sont disponibles.
Par exemple, YAFFS2 :

- Gère la flash en pages regroupées dans des blocs de 32 pages ;
- Scanne les blocs lors du montage du système de fichiers pour reconstruire la structure en mémoire (peut prendre plusieurs secondes pour les gros volumes) ;
- Utilise des identifiants d'objet et des numéros de séquence pour ne garder que la version la plus récente de chaque page ;
- Peut regrouper des pages valides dans un nouveau bloc afin de libérer les anciens blocs si l'espace libre devient restreint.

La mise en œuvre d'un tel système de fichier est en général bien plus complexe que l'utilisation de FAT mais permet l'utilisation efficace des flashes (notamment des flashes NAND).

VII. Composition

Pour exécuter du code stocké en mémoire flash, il faut qu'il soit directement accessible par le processeur. Pour cela, il faut l'une des conditions suivantes :

- Le recopier en RAM après un décodage éventuel ;
- Qu'il se trouve sous forme non compressée et non chiffrée dans une flash NOR (interne ou externe) mappée dans l'espace mémoire du processeur.

Un corollaire est que la confidentialité (par chiffrement), la faible consommation d'espace logique (par compression) ou physique (par utilisation de flash NAND) augmentent nécessairement les besoins en mémoire vive.

VII. Composition

d) Le format SquashFS

SquashFS est un système de fichier pour Linux qui :

- Permet uniquement la lecture ;
- Comprime les fichiers, répertoires et i-nodes (nœud d'index : structures de données contenant des informations concernant les fichiers stockés dans certains systèmes de fichiers);
- Est combinable avec un système de fichier en mémoire, pour obtenir une version en lecture/écriture ;
- Est intégré dans le noyau depuis la version 2.6.29 et donc peut être facilement utilisé comme système de fichiers initial ou unique.

VII. Composition

9) Réseau

Dans un réseau, deux systèmes reliés arrivent à communiquer du moment que les communications se font de part et d'autre selon les mêmes **protocoles réseau**. Selon le modèle OSI, les différents protocoles existants sont répartis sur **sept** niveaux, où un protocole d'un niveau donné peut être combiné avec n'importe quel protocole des niveaux situés en dessus et en dessous.

Un système d'exploitation contient typiquement plusieurs programmes nécessaires pour des échanges d'informations dans différents protocoles des niveaux **1** à **4**. Tandis que les niveaux **5** à **7** sont pris en charge par les logiciels applicatifs et les **middlewares** (logiciels dédié réseau).

VII. Composition

Pour les échanges d'informations selon les protocoles de niveau **1** et **2**, le système d'exploitation demande l'opération au matériel par l'intermédiaire d'un **pilote**, **pilote** qui peut faire partie intégrante du système d'exploitation ou être fourni par le constructeur du matériel.

Lors de l'envoi d'informations sur le réseau, un logiciel applicatif crée une information, la met en forme conformément aux protocoles des niveaux **7** à **5**, puis la transmet au système d'exploitation. Divers programmes du système d'exploitation vont découper cette information en trames, puis vont mettre en forme les trames et les envoyer conformément aux protocoles des niveaux **4** à **1**.

VII. Composition

Lors de la réception de trames depuis le réseau, divers programmes du système d'exploitation vont tenter de les décoder conformément à différents **protocoles** des niveaux **1** à **4**, puis transformer la suite de trames en un flux continu, qui sera envoyé au logiciel applicatif destinataire. Le logiciel va alors décoder le flux conformément aux protocoles de niveaux **5** à **7**. Le logiciel applicatif effectue préalablement une connexion, c'est-à-dire une liaison logique par laquelle il va s'associer avec un flux particulier.

Le choix exact des **protocoles** utilisés dépend du système concerné et des **liaisons réseau** qui vont être utilisées. Divers paramètres de configuration permettent d'influencer le choix des protocoles. Ils permettent par exemple d'empêcher l'utilisation de protocoles interdits sur le réseau concerné.

VII. Composition

10) Contrôle d'accès

Dans un système d'exploitation multi-utilisateurs, un ensemble de programmes assure la sécurité des informations enregistrées dans les mémoires ainsi que la disponibilité du système, ceci afin de préserver la confidentialité et éviter que des manipulations effectuées par un utilisateur perturbent l'utilisation du système par les autres utilisateurs. Cette fonctionnalité est souvent utilisée pour les ordinateurs mais est souvent absente des systèmes embarqués.

VII. Composition

11) Interface utilisateur

Suivant les systèmes, l'interface utilisateur peut être plus ou moins fournit.

Pour la partie renvoie d'information du système, cela peut aller de la **simple LED** à un **afficheur**. Un ensemble de programmes du système d'exploitation reçoit les informations envoyées par les logiciels applicatifs, et, après traitement va les retourner sur **les périphériques de sortie par l'intermédiaire de pilotes**.

Pour la partie prise d'information, cela peut aller d'un **simple bouton** à un **clavier** en passant par tous les capteurs existant. Ici, un autre ensemble de programmes reçoit les manipulations effectuées par l'utilisateur par l'intermédiaire de pilotes puis les transmettent au logiciel concerné.

Ces deux ensembles créent **l'interface homme-machine** qui permet à un usager de dialoguer avec la machine.

VII. Composition

12) Logiciels utilitaires

Un logiciel applicatif sert à assister l'utilisateur dans une activité. Les logiciels utilitaires sont des logiciels applicatifs qui permettent à l'utilisateur d'effectuer des manipulations basiques telles que démarrer des programmes, copier des fichiers ou modifier des paramètres de configuration. Divers logiciels utilitaires sont fournis avec les systèmes d'exploitation.

Certains programmes permettent à l'utilisateur à modifier **les paramètres de configuration** du système d'exploitation. Ceux-ci proposent des listes à choix multiples et effectuent des contrôles de validité avant de modifier les paramètres.

VII. Composition

D'autres programmes servent à installer des logiciels, c'est-à-dire copier les fichiers dans les emplacements prévus à cet effet, et effectuer les modifications de configuration nécessaire pour rendre le logiciel opérationnel. Ces programmes peuvent aussi servir à consulter la **liste des logiciels actuellement installés**.

Un système d'exploitation multi-utilisateurs est en général fourni avec des programmes permettant de surveiller l'utilisation - par autrui - de l'ordinateur – consultation de journaux d'activité – ou de modifier les listes de droits d'accès en vue d'autoriser ou d'interdire un fichier à certains utilisateurs.

VII. Composition

13) Le noyau

Le noyau (**kernel**) est un espace **mémoire isolé**, dans lequel est placé **tout ou partie** du système d'exploitation. Dans le langage courant le terme kernel désigne l'emplacement ainsi que l'ensemble des programmes qu'il contient et qui forment le cœur rigide du système d'exploitation. Le contenu du noyau ne peut pas être modifié par inadvertance par les logiciels applicatifs.

VII. Composition

14) Organisation générale

Un système d'exploitation est typiquement organisé en **couches distinctes**.

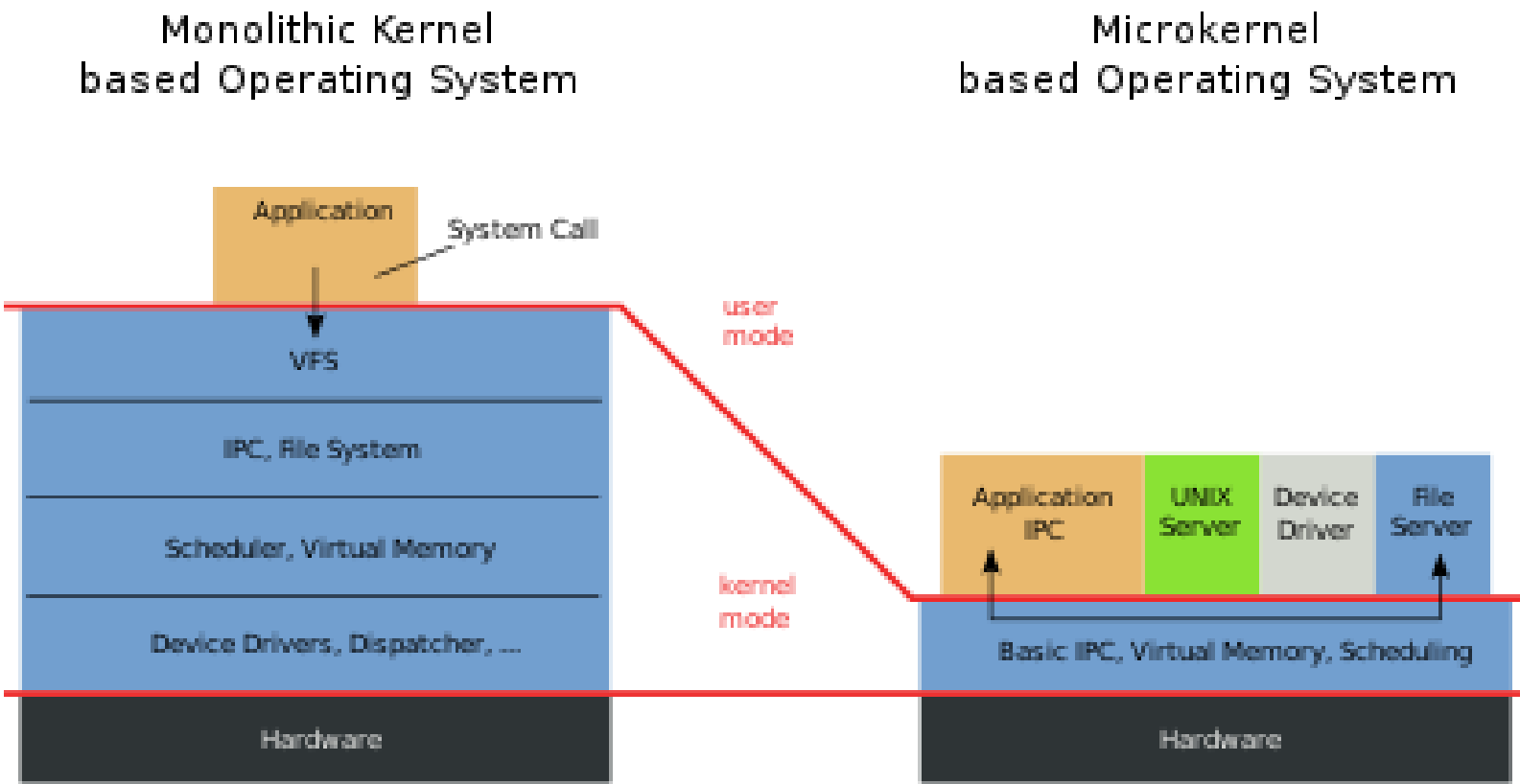
La couche supérieure est **l'interface de programmation** avec les logiciels applicatifs (dont font partie les logiciels utilitaires fournis avec le système d'exploitation).

Au centre, on trouve une ou plusieurs couches qui contiennent les composants **principaux du système d'exploitation** tels que : les programmes pour les systèmes de fichiers et le réseau, la gestion de mémoire, les pilotes, l'ordonnanceur, le gestionnaire d'interruption.

La couche inférieure, appelée **couche d'abstraction matérielle** (hardware abstraction layer abrégé HAL), est chargée de masquer les particularités matérielles et les différences qu'il existe entre les machines sur lesquelles le système d'exploitation sera utilisé.

VII. Composition

Comparaison entre noyau monolithique ou micronoyau



VIII. Architecture des systèmes d'exploitation

VIII. Architecture des systèmes d'exploitation

1) Système d'exploitation Monolithique (plus ancien)

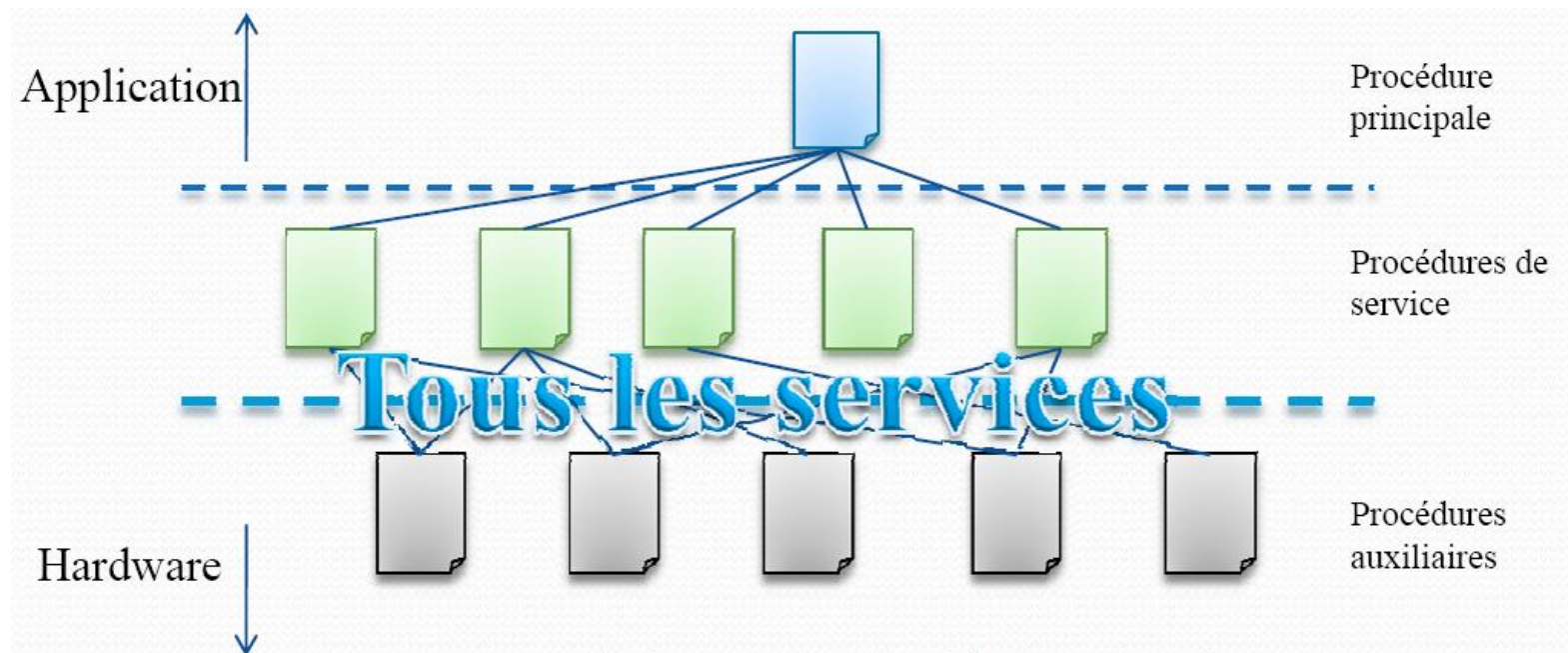
Très Simple, le système d'exploitation Monolithique ne consomme pas beaucoup de ressources. Il convient aux «**petits systèmes**» ou quelques portions de systèmes temps réel complexe. Il est entièrement en mode privilégié. L'application utilise un appel système pour accéder aux services du système d'exploitation, c'est une procédure exécutée. La gestion de l'interruption est optimisée car il n'y a pas de changement de contexte entier (prioritaire car l'ordonnanceur est désactivé). Il est impossible de mettre à jour l'application «à chaud» (remplacement + reboot).

La structure de base est composé de :

- Un programme principal qui invoque la procédure du service
- Un ensemble de procédures de services qui gèrent les appels système
- Un ensemble de procédures utilitaires auxiliaires des précédentes

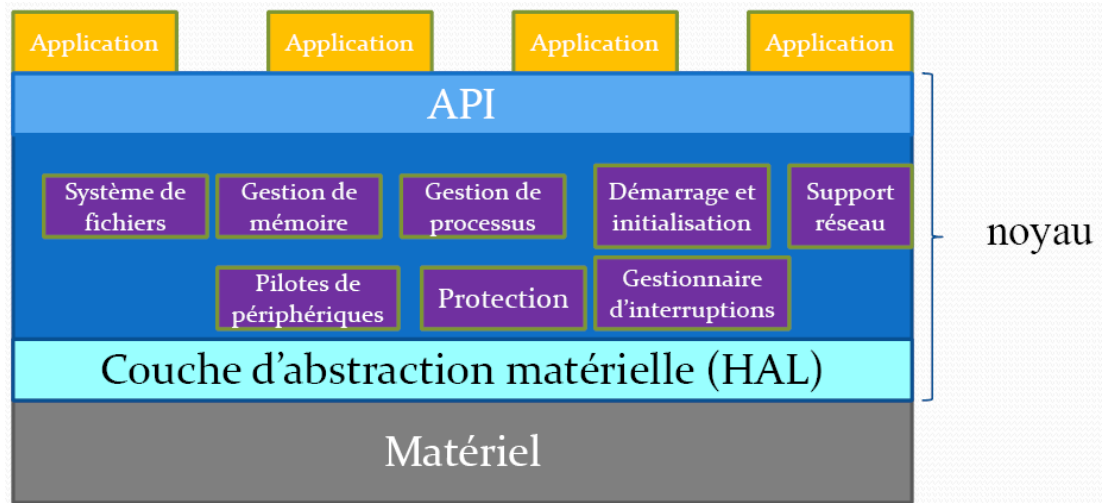
VIII. Architecture des systèmes d'exploitation

Ce type d'architecture a été utilisé pour les vieilles version d'UNIX (FreeBSD, SOLARIS) et DOS.



VIII. Architecture des systèmes d'exploitation

Exemple : Unix



Avantages :

- De meilleures performances
- Vite développé ...
- Dernières versions: chargement dynamique (et donc sélectif) des module

Inconvénients :

- Extension difficile : Code non modulaire
- Très complexe
- Code massif
- Plus c'est gros, moins c'est performant !
- Nid de bugs
- Peu fiable (redémarrage à chaque Bug)

2) Système d'exploitation Multicouches

Le système d'exploitation Multicouches est organisé en hiérarchie de couches. Chacune construite sur la base des services offerts par la couche inférieure. L'Interface et la gestion des interruptions est similaire à celle des systèmes monolithiques. Is est doté d'une meilleure structure et d'une meilleure modularité ce qui lui apporte une maintenance plus aisée. La configuration est plus fine (modularité) et donc on a une meilleure utilisation de la mémoire et des performances.

VIII. Architecture des systèmes d'exploitation

Description des différentes couches (exemple de la Structure de l'OS THE (Technische Hogeschool Eindhoven) de Dijkstra créée en 1968) :

- Couche 0: fournit le service de multiprogrammation. Dans la première couche, c'était un ensemble de processus séquentiels indépendant.
- Couche 1: Allocation de mémoire dans la mémoire principale. Au-dessus, les processus ignoraient les emplacements mémoire.
- Couche 2: gestion de la communication entre processus et la console opérateur. Dans la couche précédente, chaque processus avait sa propre console opérateur.
- Couche 3: périph d'E/S et mise en mémoire tampon des flots d'E/S. Au-dessus, les processus travaillaient sur une couche abstraite.
- Couche 4: contient les programmes utilisateur, les notions de processus, mémoire, console leurs sont abstraites.
- Couche 5: le processus opérateur (control global du système).

VIII. Architecture des systèmes d'exploitation

Couche	Fonction
5	Opérateur
4	Programmes utilisateur
3	Gestion des E/S
2	Communication opérateur-processus
1	Gestion de la mémoire et du tampon
0	Allocation du processeur-multiprogrammation

Avantages :

- Facile à étendre (plus structuré)
- Modèle simple

Inconvénients :

- Traverser les différentes couches peut être coûteux
- La multiplication des couches peut ne pas être nécessaire.
- Performances moins bonnes (monolithique)

VIII. Architecture des systèmes d'exploitation

3) Système d'exploitation Micronoyau

Plusieurs fonctions du l'Système d'exploitation Micronoyau est déplacé vers des «**processus serveur**» s'exécutant en mode utilisateur ce qui permet une réduction au maximum de la taille du code privilégié.

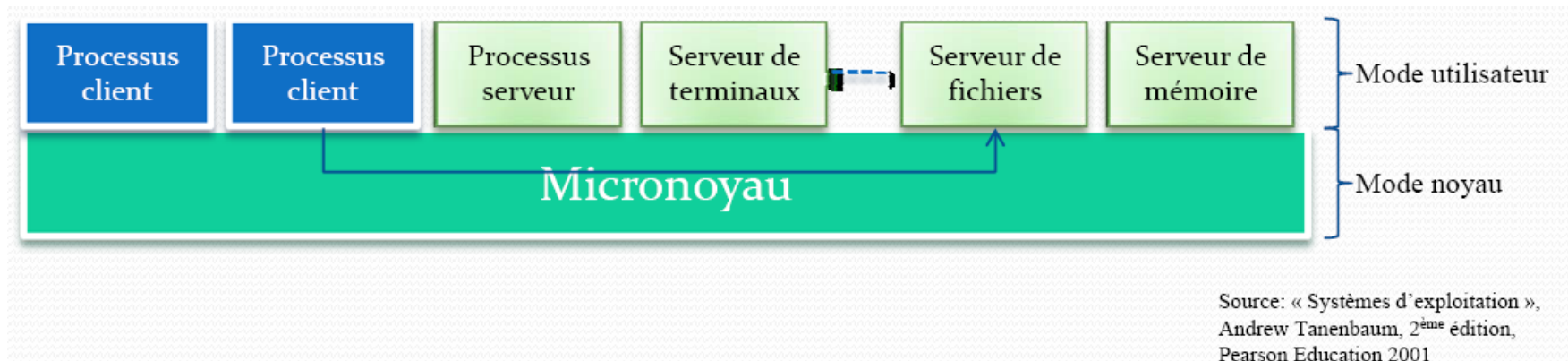
Le but de cette architecture est de gérer les communications entre applications et serveurs pour renforcer la politique de sécurité et permettre l'exécution de fonctions système (accès aux registres d'E/S, etc.).

Il a une meilleure fiabilité : Si un processus serveur «crash», le système continue à fonctionner et il est possible de relancer ce service sans redémarrer.

Ce modèle est facilement étendu à des systèmes distribués.

VIII. Architecture des systèmes d'exploitation

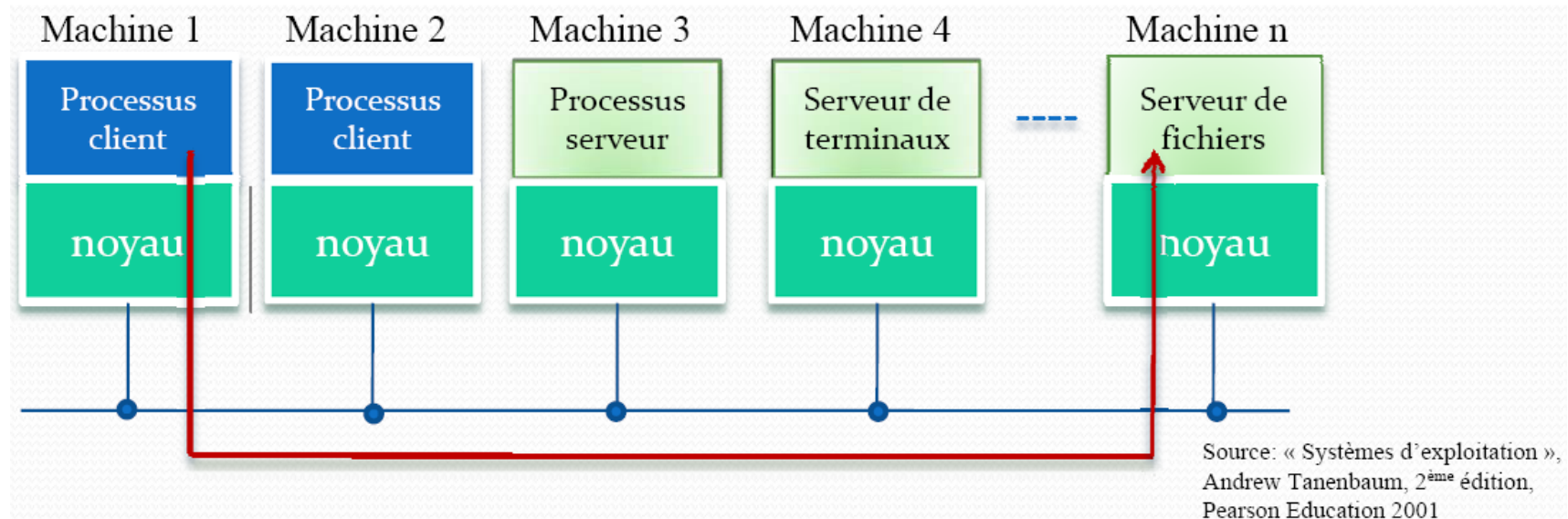
La gestion de l'interruption se fait par commutation de tâche, ce qui est moins efficace que le modèle monolithique.



Le noyau gère les communications entre clients et serveurs.

Certains services sont impossibles à exécuter en mode utilisateur (pilotes de périphériques d'E/S).

VIII. Architecture des systèmes d'exploitation



Si le client communique avec le serveur par envoi de message, il lui importe peu que le serveur soit local ou distant, le résultat est le même d'où l'adaptabilité aux systèmes distribués

Exemples: Windows NT, Mach, Chorus, QNX

VIII. Architecture des systèmes d'exploitation

Avantages :

- Extensibilité
- Minimise le code du noyau
- Sécurité: Un serveur crash, il sera le seul à redémarrer
- Fiabilité: code plus petit donc moins de bugs

Inconvénients :

- Souvent tenté de rajouter des choses dans le noyau (vu qu'il est petit...)
- Mauvaises performances
- Requièrè beaucoup de prudence lors de la conception

4) Système d'exploitation Machine Virtuelle

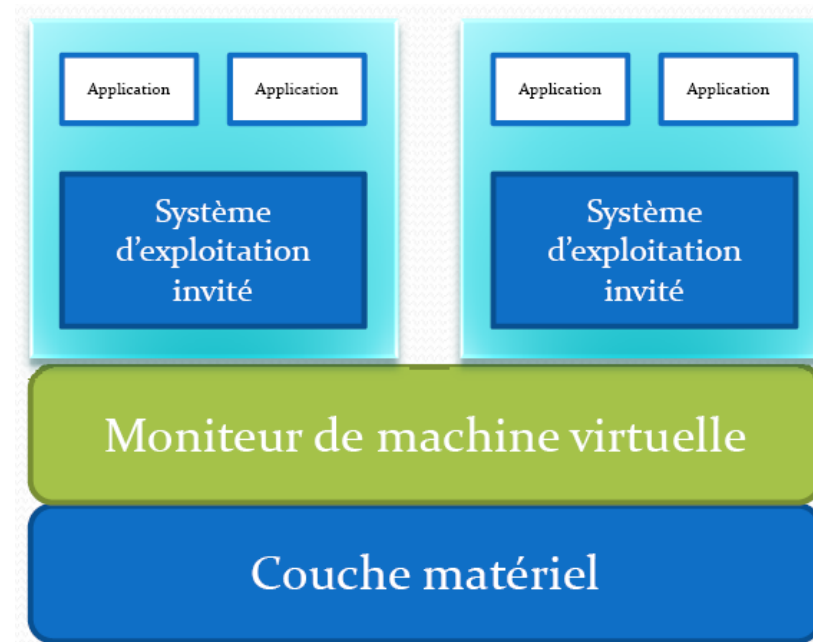
Le système d'exploitation Machine virtuelle doit remplir 2 fonctions:

- Multi programmation
- Services système (Un ou plusieurs OS «invités» qui s'exécutent sur les processeurs virtuels et fournissent les services système)

2 types de système d'exploitation Machine Virtuelle:

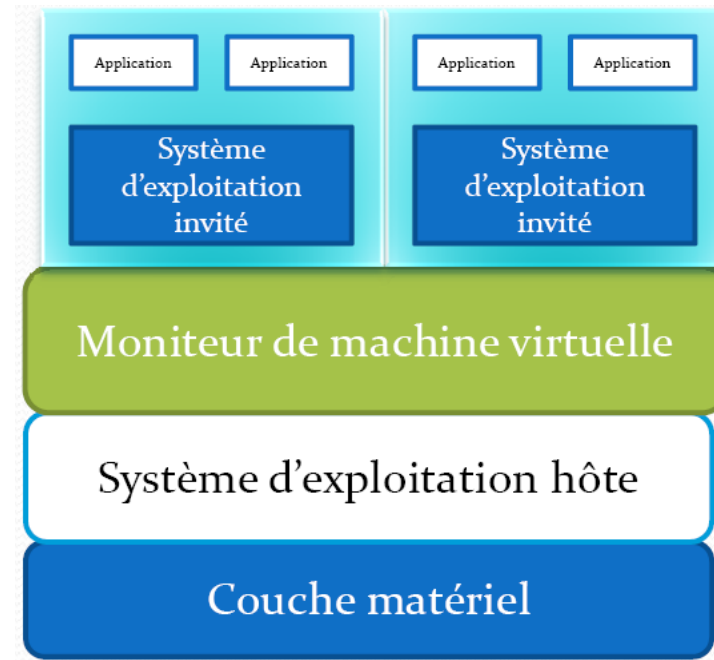
VIII. Architecture des systèmes d'exploitation

- MV native



VIII. Architecture des systèmes d'exploitation

- MV invité



VIII. Architecture des systèmes d'exploitation

Exemples de ce type de système d'exploitation: XEN, VMWare, IBM' VM/370.

Avantages :

- Permet l'exécution de plusieurs OS sur une seule machine
- Permet une bonne portabilité des applications
- Une protection complète (code exécute en mode privilégié complètement géré)
- Bon environnement de développement (devsystème en mode utilisateur...)

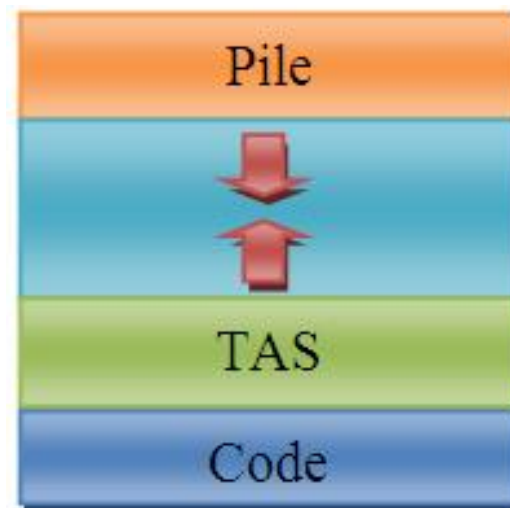
Inconvénients :

- Gros problème de performances (plusieurs couches)
- Manque de flexibilité

IX. Principes de base des tâches Processus et Threads

IX. Principes de base des tâches Processus et Threads

Un processus est un ensemble d'activités corrélées ou interactives qui transforment les éléments d'entrée en éléments de sortie. Pour les systèmes d'exploitation embarqués est donc un programme en exécution



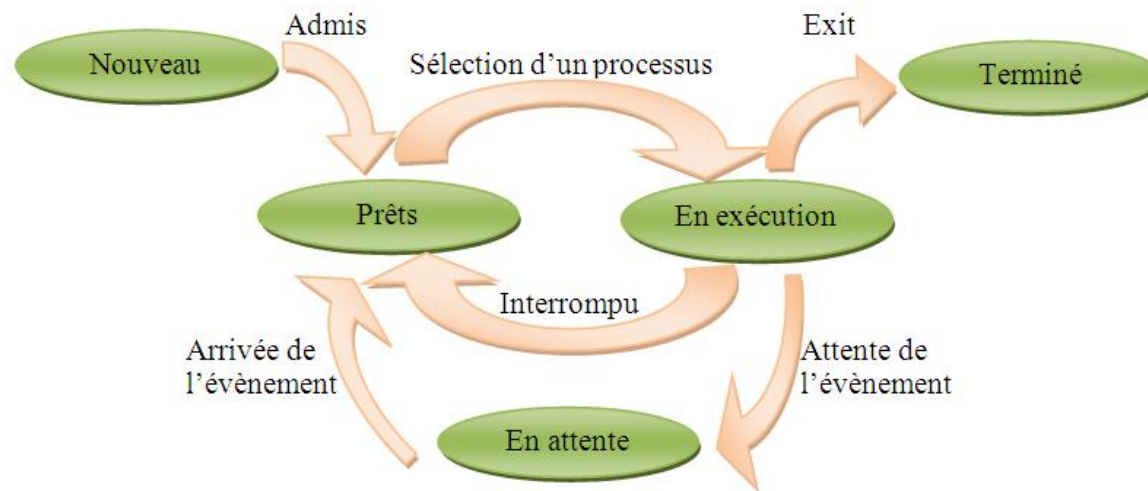
IX. Principes de base des tâches Processus et Threads

Il comporte :

- Le code: programme
- Les données: variables globales et dynamiques (appelé aussi tas)
- La pile d'exécution : variables locales, mise en œuvre des appels et retour des fonctions
- Le contexte : les registres internes (IP, SP...), la table des fichiers ouverts, les variables d'environnement comme errno, CLASSPATH...

La pile et le tas ont des frontières mobiles qui croissent en sens inverse.

IX. Principes de base des tâches Processus et Threads



- **Nouveau** : le processus est en cours de création
- **En exécution (Running)** : les instructions sont en cours d'exécution par le CPU
- **En attente (Sleep)** : le processus attend qu'un événement se produise événement : un signal, terminaison d'une E/S, libération d'une ressource...
- **Prêt (Ready)** : le processus attend d'être affecté au CPU
- **Terminé** : le processus a fini son exécution

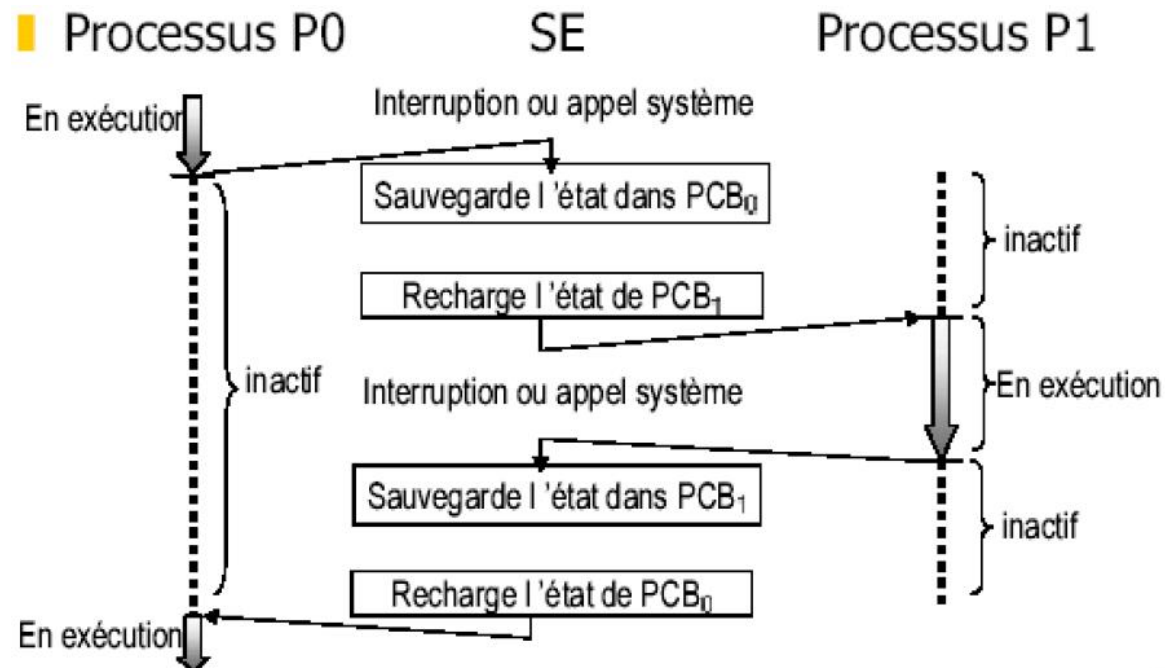
IX. Principes de base des tâches Processus et Threads

Chaque processus est représenté dans le SE par un bloc de contrôle (Process Control Bloc : PCB) constitué de :

- L'état du processus : nouveau, prêt, en exécution
- Le compteur d'instructions : prochaine instruction à exécuter
- Les registres du CPU
- Date de création du processus
- Informations sur son numéro (PID)
- Informations sur le répertoire de travail
- Informations de l'identité de l'utilisateur, de son groupe (UID, GID)
- Informations sur l'état des E/S (ex : liste des fichiers ouverts)
- Etc...

IX. Principes de base des tâches Processus et Threads

La commutation entre 2 processus s'illustre de la manière suivante :



IX. Principes de base des tâches Processus et Threads

Les inconvénients des Processus sont:

- Ils sont Lourds en mémoire (duplication du code)
- La communication entre processus difficile (segments séparés).

Un thread (ou processus léger) est similaire à un processus, sauf au point de vue gestion de la mémoire. Là où chaque processus possède sa propre mémoire virtuelle, les threads d'un même processus se partagent sa mémoire virtuelle. Par contre, tous les threads possèdent leur propre pile d'appel.

Dans un même processus, on peut avoir plusieurs threads qui aura chacun sa piles et un PCB réduit permettant la sauvegarde des registres, mais le code, le contexte et tas sont partagés.

IX. Principes de base des tâches Processus et Threads

Les «petits» systèmes d'exploitation embarqués utilisent seulement les threads alors que les «grands» systèmes d'exploitation peuvent utiliser plusieurs modèles processus/threads.

Il existe 2 types de Processus et de threads :

- Statique: toutes les tâches sont connues à l'avance. Il n'est pas possible d'en créer pendant que le système tourne.
- Dynamique: appels système permettant de créer et détruire des tâches à la volée: Système plus flexible mais plus de complexité (allocation dynamique, gestion d'erreurs)

X. Les normes des systèmes d'exploitation

X. Les normes des systèmes d'exploitation

1) POSIX (Portable Operating System Interface)

Standard pour les **appels de fonction** (API) pour les OS UNIX-like. Il existe quelques spécifications pour des primitives temps réels.

4 profils pour le temps réel:

- PSE51: **profile de système temps réel minimaliste : 1 seul processus POSIX pouvant exécuter plusieurs threads POSIX pouvant utiliser le passage de messages POSIX pour communiquer avec d'autres systèmes PS5xHw: 1 seul processeur avec sa mémoire, pas de MMU et d'E/S standard.**
- PSE52: **profile de système de contrôleur temps réel: PSE51+support pour un système de fichiers + E/S asynchrones**
- PSE53: **profile de système temps réel dédié: PSE51+support multiprocessus (+MMU)**
- PSE54: **profile de système temps réel polyvalent: englobe les autres profils. Il consiste de POSIX.1, POSIX.1b, POSIX.1c, et/ou POSIX.5b**

X. Les normes des systèmes d'exploitation

Exemple: RTLinuxse réclame du profil PSE51 alors qu'RTAI ne réclame rien du tout.

X. Les normes des systèmes d'exploitation

2) UNIX98

Normalisation de l'OS UNIX. Cette norme incorpore plusieurs des normes de POSIX

X. Les normes des systèmes d'exploitation

3) EL/IX

API pour les systèmes embarqués. Se veut un sous ensemble des normes POSIX et ANSI.

X. Les normes des systèmes d'exploitation

4) μ TRON

Norme japonaise pour les systèmes embarqués

X. Les normes des systèmes d'exploitation

5) OSEK

Norme allemande pour une architecture ouverte reliant les divers contrôleurs électroniques d'un véhicule.

OSEK est le sigle pour « Offene Systeme und deren Schnittstellen für die Elektronik im Kraftfahrzeug », en français **Systèmes ouverts et interfaces correspondantes pour l'électronique des véhicules automobiles**.

OSEK a été créé en 1993 par un consortium de constructeurs et équipementiers automobiles allemands (BMW, Bosch, DaimlerChrysler, Opel, Siemens, et VW) ainsi qu'un département de l'université de Karlsruhe. Leur but était de développer un standard pour une architecture ouverte reliant les divers contrôleurs électroniques d'un véhicule. En 1994, les constructeurs français Renault et PSA qui développaient un projet similaire, VDX (Vehicle Distributed eXecutive), rejoignirent le consortium.

X. Les normes des systèmes d'exploitation

6) RT Spec pour Java

Spécification pour un runtime qui édicte des prescriptions (ramasse miettes, certaines politiques d'ordonnancement, etc.)

X. Les normes des systèmes d'exploitation

7) RT Corba

Un ensemble de spécification temps réel

XI. Logiciel Libre et systèmes embarqués

XI. Logiciel Libre et systèmes embarqués

Un logiciel libre est un logiciel dont l'utilisation, l'étude, la modification et la duplication en vue de sa diffusion sont permises, techniquement et légalement, afin de garantir certaines libertés à l'utilisateur.

Le logiciel libre s'impose de plus en plus comme une solution de remplacement moins coûteuse de logiciels propriétaires.

Ces droits peuvent être simplement disponibles (cas du domaine public) ou bien établis par une licence, dite « libre », basée sur le droit d'auteur. Les « licences copyleft » garantissent le maintien de ces droits aux utilisateurs même pour les travaux dérivés. Les logiciels libres constituent une alternative à ceux qui ne le sont pas, qualifiés de « propriétaires » ou de « privateurs ».

XI. Logiciel Libre et systèmes embarqués

Un logiciel est libre selon la Free Software Foundation s'il confère à son utilisateur quatre libertés :

- La liberté d'exécuter le programme, pour tous les usages,
- La liberté d'étudier le fonctionnement du programme et de l'adapter à ses besoins,
- La liberté de redistribuer des copies du programme (ce qui implique la possibilité aussi bien de donner que de vendre des copies),
- La liberté d'améliorer le programme et de distribuer ces améliorations au public, pour en faire profiter toute la communauté.

XI. Logiciel Libre et systèmes embarqués

« GPL » est l'acronyme de « General Public License » (Licence Publique Générale). La plus répandue des licences de ce type est la Licence Publique Générale GNU (GNU General Public License), ou GNU GPL pour faire court. On peut réduire encore plus en « GPL », s'il est entendu qu'il s'agit de la GNU GPL.

Certaines licences, dont la plus connue et utilisée pour les logiciels libres, la licence publique générale GNU, sont relativement complexes. Ainsi, la GPL ne donne le droit de redistribuer un logiciel que si l'ensemble du logiciel, y compris toutes les éventuelles modifications, sont redistribués selon les termes exacts de la GPL. Cette licence est dite « virale » ou « contaminante » par ses opposants, car si elle autorise la fusion d'un logiciel sous GPL avec un logiciel sous une autre licence, elle n'autorise en revanche la redistribution du logiciel fusionné que sous GPL.

XII. Exemple de système d'exploitation

XII. Exemple de système d'exploitation

Il existe beaucoup de systèmes d'exploitation embarqué, mais ils sont souvent dédiés à des applications dédiées (Téléphonie, PC, Recherche, Spatial, Automobile, Aviation...)

XII. Exemple de système d'exploitation

1) PalmOS

Palm OS est un système d'exploitation embarqué et propriétaire permettant le fonctionnement d'appareils électroniques (téléphone, PDA,...). Conçu en 1996 pour le nouveau PDA Pilot de Palm Computing, Inc. , il a été implémenté sur une large gamme d'appareils électroniques mobiles, dont des Smartphones, montres bracelet, consoles de jeux portables, lecteur de code-barres et appareils GPS.

Les versions de Palm OS antérieures à la 5.0 fonctionnent sur processeurs Motorola/Freescale DragonBall. Depuis la version 5.0, Palm OS fonctionne sur processeurs fondés sur l'architecture ARM.

Aujourd'hui, après avoir été renommée Garnet OS, intégré dans Linux embaqué et plusieurs rachats, de Palm OS survit dans Palm WebOS

XII. Exemple de système d'exploitation

Les fonctionnalités principales de la version actuelle de Palm OS (Garnet 5.4) sont:

- Environnement à tâche unique et simple pour permettre le lancement d'applications en plein écran avec un ensemble commun et de base pour l'interface graphique
- Support d'écrans monochromes ou couleurs avec des résolutions jusqu'à 480x320
- Système d'entrée de données par reconnaissance de l'écriture manuscrite appelé Graffiti 2
- Technologie HotSync pour la synchronisation avec les ordinateurs de bureau
- Capacités d'enregistrement et de lecture du son
- Modèle de sécurité simple: L'appareil peut être verrouillé par mot de passe, les enregistrements dans les applications peuvent être rendus privés de manière arbitraire

XII. Exemple de système d'exploitation

- Accès au réseau utilisant le protocole TCP/IP
- Connexions par ports port série/USB, Infrarouge, Bluetooth et WiFi
- Support d'un port d'extension pour carte mémoire
- Format de données standard défini pour les applications de gestionnaire d'informations personnelles qui stockent les entrées de l'agenda, du carnet d'adresses, des tâches à réaliser et des notes, accessibles par des applications d'éditeurs tiers.
- D'autres applications standards.

XII. Exemple de système d'exploitation

Ce système d'exploitation offre :

- Une gestion de la mémoire simplifiée
- Des primitives de gestion de bases de données et de l'écran
- Des bibliothèques mathématiques
- Des applications minimalistes (philosophie du Palm)

Il est mono-application et mono-thread et donc il ne nécessite pas beaucoup de puissance (m68k 20MHz, ARM à 200MHz en émulation m68k)

XII. Exemple de système d'exploitation

Le système d'exploitation a été amélioré et utilisé par d'autres sociétés pour leurs propres plateformes sous licence :

- Handspring (séparé de Palm puis racheté par PalmOne), pour ses PDA Visor et ses téléphones Treo
- Sony, pour ses PDA Clié
- IBM, pour ses PDA Workpad
- Qualcomm, pour sa gamme de Smartphone
- Symbol, pour une gamme de scanner de code-barres intelligents
- Tapwave, pour une console de jeux portable « Zodiac »
- Fossil, pour une montre

XII. Exemple de système d'exploitation

- Fossil, pour une montre
- Garmin, pour un GPS
- HandEra, pour ses PDA TRGpro et Handera
- Acceca
- AlphaSmart
- Samsung
- GSPda

On distingue deux types de machines : les PDA « classiques » et les machines hybrides PDA+Téléphone appelées « Smartphone ».

XII. Exemple de système d'exploitation

2) Windows CE

Windows CE (officiellement nommé Windows Embedded Compact depuis la version 6.0), parfois abrégé WinCE, est une variation de Windows pour les systèmes embarqués et autres systèmes minimalistes, utilisée notamment dans les PC de poche ou Handheld. Il utilise un noyau distinct des autres Windows plutôt qu'une version allégée.

Il est doté des fonctionnalités générales suivantes :

- Windows CE est prévu pour fonctionner sur des périphériques à **faible** empreinte mémoire. La taille d'une image système sans support graphique peut être de l'ordre du **Mo**.
- Windows CE peut fonctionner **sans support de stockage, exclusivement en RAM/ROM**

XII. Exemple de système d'exploitation

- Windows CE est un système 32 bits pouvant fonctionner sur architecture ARM, x86, MIPS, Hitachi SH.
- Windows CE répond aux contraintes d'un système temps-réel avec un ordonnanceur préemptif à 256 niveaux de priorités fixes et un temps de latence des interruptions déterministe. Sous certaines conditions, un mécanisme d'inversion de priorité peut être pris en charge par l'OS. L'unité d'exécution est le thread.
- Windows CE prend en charge un environnement graphique basé sur une mécanique événementielle (GWES - Graphic Windowing and Events Subsystem) et intégrant une couche GDI (Graphic Device Interface) allégée par rapport à celle disponible sur XP.
- Communication: Bluetooth, IrDA, Ethernet, Wifi, NDIS 5.1, TCP/IP, ...

XII. Exemple de système d'exploitation

Il intègre donc tous les services d'un Windows, mais également la mauvaise fiabilité de ceux-ci, mais permet une facilité de portage des applications.

Il nécessite un processeur très puissant (ex : ARM 400MHz).

Les outils de développements se répartissent en trois familles :

- Développement système : Platform Builder.
- Développement applicatif : Embedded VC, Visual Studio et WINDEV Mobile.
- Diagnostic et analyse : Remote tools.

Il existe également un compilateur croisé libre qui permet de développer des applications Windows CE à partir d'autres plateformes comme Linux, FreeBSD ou Mac OS X : CeGCC (dérivé de la famille des GCC).

XII. Exemple de système d'exploitation

Une licence de développement, sous forme d'un outil de génération d'images (Platform Builder), est nécessaire. Les images générées ne sont alors plus limitées à 120 jours. Par ailleurs, chaque appareil livré doit être doté d'une licence (Run time License), matérialisée par un autocollant. Aucune activation n'est prévue. Les licences peuvent être acquises auprès de distributeurs spécifiques et agréés par Microsoft, après signature d'un contrat d'acceptation des conditions.

Anecdote : Bien que le système d'exploitation principal de la console de jeux vidéo Dreamcast de SEGA fût SegaOS, la Dreamcast était également compatible avec Windows CE. Bien que peu utilisé, celui-ci facilitait notamment le portage de jeux PC DirectX vers la console en un temps record. Il fut notamment utilisé pour le jeu [Sega Rally 2](#). Cependant l'utilisation de Windows CE n'était pas obligatoire. En effet, la console utilisait SegaOS par défaut, et le système était éventuellement chargé si le jeu en question était un jeu Windows CE (identifié par la présence du logo Powered by Windows CE, notamment au lancement du jeu), directement depuis le GD-ROM du jeu, la Dreamcast ne possédant pas Windows CE sur mémoire morte.

XII. Exemple de système d'exploitation

3) Symbian OS

Symbian OS est un système d'exploitation multitâches préemptif pour téléphones portables de type Smartphone et de PDA. Il est né d'un consortium entre les différents constructeurs (initialement Psion, Nokia, Motorola, Ericsson et Matsushita/Panasonic puis également Siemens et Samsung). Il dispose de nombreuses API spécifiques pour la communication mobile voix et données, et utilise des protocoles standards de communication réseaux: IPv4/IPv6, WAP, MMS, Bluetooth, GPRS/UMTS, Java, SyncML... Le 16 novembre 2006, 100 millions de téléphones mobiles avaient été vendus avec ce système d'exploitation.

Il a été racheté à 100 % par Nokia, alors que le géant finlandais ne détenait que 48 %, et que le reste était détenu par Sony Ericsson, Siemens, Samsung et Panasonic. À la suite de ce rachat Nokia a décidé de changer la licence de Symbian OS qui est Open Source depuis le 21 octobre 2009 et officiellement téléchargeable depuis le 4 février 2010. Le 11 février 2011, Nokia annonce l'abandon de Symbian pour le remplacer par Windows phone 7. Il nécessite en revanche moins de ressources que Windows CE

XII. Exemple de système d'exploitation

4) Android

Android est un système d'exploitation open source pour Smartphones, PDA et terminaux mobiles conçu par Android, une startup rachetée par Google. D'autres types d'appareils possédant ce système d'exploitation existent, comme par exemple des téléviseurs, des autoradios et des tablettes.

Android est un système d'exploitation fondé sur un noyau Linux, il comporte une interface spécifique, développée en java, les programmes sont exécutés via un interpréteur JIT, toutefois il est possible de passer outre cette interface, en programmant ses applications en C, mais le travail de portabilité en sera plus important. Si la majorité des périphériques Android sont basés sur l'architecture ARM, l'hétérogénéité des versions et des coprocesseurs peut varier grandement d'un constructeur à l'autre.

Disponible via une licence Apache version 2, le système d'exploitation inclut tous les utilitaires requis par un constructeur ou par un opérateur pour mettre en œuvre un téléphone portable.

XII. Exemple de système d'exploitation

5) Linux embarqué

Linux embarqué (**embedded linux**) désigne un système d'exploitation basé sur Linux et adapté à un système embarqué. Ils sont multitâches préemptif et multi-utilisateur.

Il existe de nombreuses versions de linux embarqués adaptés à différents types de matériels (téléphones portables, set-top boxes, boitiers ADSL, lecteurs multimédia, etc).

Contrairement aux versions de Linux destinées aux ordinateurs personnels et aux serveurs, les différents systèmes linux embarqués sont conçus pour des systèmes aux ressources limitées mais tout en gardant la fiabilité des versions complètes.

XII. Exemple de système d'exploitation

Les systèmes embarqués sous Linux disposent généralement de peu de RAM et utilisent fréquemment de la mémoire flash plutôt qu'un disque dur. Comme ils sont souvent dédiés à un nombre de tâches réduites sur une cible matérielle bien définie, ils utilisent plutôt des versions du noyau Linux optimisée pour des contextes précis.

Les distributions Linux destinées à l'embarqué disposent généralement d'un noyau temps réel.

XII. Exemple de système d'exploitation

6) RTEMS

RTEMS (**Real-Time Executive for Multiprocessor Systems**) est un système d'exploitation temps réel libre, multitâches préemptif pour systèmes embarqués.

Il permet de développer des programmes dans lesquels le temps de réponse et la réactivité sont des contraintes fortes (**dits temps-réel dur**). Il a été porté sur de nombreux processeurs (ARM, i386, m68k, MIPS, PowerPC, SH, etc.), et est compatible avec divers standards de programmation ouverts comme POSIX et uITRON, ainsi qu'avec la pile TCP/IP de FreeBSD et plusieurs systèmes de fichiers. Il ne nécessite pas beaucoup de puissance (ex : m68k à 33Mhz)

Il est très utilisé dans le milieu expérimentations physiques, notamment en milieu spatial (utilisé dans le projet Mars Reconnaissance Orbiter)

XII. Exemple de système d'exploitation

7) FreeRTOS

FreeRTOS est un noyau de système d'exploitation temps réel multitâches préemptif pour microcontrôleur. Il est distribué sous la licence libre GPL avec une exception facultative. L'exception permet aux utilisateurs avec du code source propriétaire de garder leur code source fermé/secret tout en maintenant le noyau lui-même en logiciel libre (open source), ce qui facilite l'utilisation de FreeRTOS dans des applications propriétaires.

XII. Exemple de système d'exploitation

Architectures matérielles supportées :

- ARM ARM7, ARM9 et ARM Cortex-M3 / M4
- Atmel AVR et AVR32
- MSP430
- Microcontrôleur PIC PIC18, PIC24, dsPIC, PIC32
- Renesas H8/S, SuperH, RX600
- x86
- 8052
- Motorola ColdFire V1 et V2
- Fujitsu MB91460 series et MB96340
- ...

XII. Exemple de système d'exploitation

FreeRTOS est conçu pour être petit (**très peu d'utilisation mémoire**) et simple. Le noyau lui-même se compose que de trois ou quatre fichiers en langage C. Pour rendre le code plus lisible, facile à porter et à maintenir, il est écrit principalement en C, mais il y a quelques fonctions en langage assembleur qui sont incluses où elles sont nécessaires (la plupart du temps dans les routines de l'ordonnanceur qui sont spécifique à chaque architecture).

Principales caractéristiques:

- **Petit (très peu d'utilisation mémoire) et simple. Très bon pour les débutants/amateurs qui débutent sur les systèmes d'exploitation temps réel pour système embarqué.**
- **L'ordonnanceur peut être configuré pour fonctionner mode préemptif ou coopératif.**
- **Le support de coroutine (la coroutine dans FreeRTOS est une tâche très simple et légère qui a une utilisation très limitée de la pile)**

XII. Exemple de système d'exploitation

8) VxWorks

VxWorks est un système d'exploitation temps réel multitâche, généralement utilisé dans les systèmes embarqués.

Ce système d'exploitation de la firme Wind River (acquise par Intel en 2009), a été employé par la NASA pour les missions spatiales Mars Pathfinder, Stardust, les rovers martiens (Spirit et Opportunity) et la sonde Mars Reconnaissance Orbiter. Il est principalement utilisé par la recherche et l'industrie, mais également dans de nombreux systèmes de communication d'entreprise (IP PBX Mitel ICP de Mitel Networks, Nortel Communication Servers 1000[]). Ce système peut gérer plusieurs cartes mères, chacune possédant à son tour des slots d'extension afin d'y ajouter des interfaces de toutes sortes (sondes de températures, réseau, écran, cartes d'entrées/sorties).

XII. Exemple de système d'exploitation

Les langages utilisés avec ce système d'exploitation sont le C, le C++ et l'Ada (AdaCore, qui développe le GNAT Pro, a basé le tasking Ada sur VxWorks pour différentes cibles).

Il inclut en natif un support TCP/IP, mais le coût de la licence est assez important.

XII. Exemple de système d'exploitation

9) QNX

QNX (prononcé Q-N-X ou Q-nix) est un système d'exploitation UNIX commercial temps réel compatible POSIX, conçu principalement pour le marché des systèmes embarqués. Il utilise un micronoyau temps réel.

Dans le cas de QNX, l'utilisation d'un micronoyau permet aux utilisateurs, ou plus particulièrement aux développeurs, de désactiver les fonctionnalités qui ne leur sont pas nécessaires, sans avoir à changer de système d'exploitation.

Il est également considéré à la fois comme léger, robuste (micronoyau), rapide et complet et de plus gratuit pour des applications non commerciales. Grâce à sa compatibilité POSIX, de nombreuses applications open source ont été portées sur QNX.